教案例子 · 进阶：条件扩散模型

MNIST 手写数字 · 类别编码为条件 · Classifier-Free Guidance

前置：《2D DDPM 动手实验》教案 | 配套数学：《扩散生成模型的数学原理》

课题	条件 DDPM：把数字类别 $c \in \{0, \dots, 9\}$ 编码进 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c)$，按需生成指定数字
对象	已完成双月无条件实验的本科高年级 / 研究生
先修	双月教案（前向闭式、$\boldsymbol{\epsilon}$-loss、祖先采样）；PyTorch 卷积基础；讲义第二、四章
时长	90–120 分钟（概念 20 + 代码对照 25 + 训练等待/讲解 30 + 采样与 CFG 讨论 20）
硬件	课堂 GPU：A100 / H200（单卡即可）。`classroom_cond_mnist.py` 10 epochs 估计 2–5 分钟（未在 GPU 上实测，脚本会打印真实耗时）；参考：8 核 CPU 实测 8 epochs 27.5 分钟。无时间时直接加载 `runs/final/ckpt_ema.pt`
产出	0–9 条件采样网格图、CFG 对比图、loss 曲线；能口述「条件如何进入网络」

一、与双月实验的关系

	双月实验（入门）	本实验（进阶）
数据	2D 点云 `make_moons`	28×28 灰度图 MNIST
网络	小 MLP	U-Net + 7×7 自注意力（约 214 万参数）+ EMA
条件	无（只学 $p(\mathbf{x})$）	有：类别 c（学 $p(\mathbf{x} \mid c)$）
采样控制	只能「随机生成一个双月」	可指定「生成数字 3」
新概念	—	类别嵌入、CFG（可选）

数学骨架不变：前向闭式、$\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta\|^2$、反向均值公式。变的是网络输入多了 c，以及采样时可选用 guidance。

二、教学目标

1. 说清「条件」的含义：网络从 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ 变成 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c)$，学的是条件分布 $p(\mathbf{x}_0 \mid c)$；
2. 指出类别嵌入在代码中的位置，并解释为何把它与时间嵌入相加后注入 ResBlock；
3. 完成带标签的训练循环（含随机丢弃标签以支持 CFG），并按指定数字采样；
4. （可选）解释 Classifier-Free Guidance：$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\emptyset + w(\boldsymbol{\epsilon}_c - \boldsymbol{\epsilon}_\emptyset)$，并观察 w 对样本的影响。

三、数学要点（条件版）

3.1 条件前向与训练

前向过程不依赖条件（噪声日程与数据标签无关）：

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{1}$$

训练时，每个样本带着它的标签 c（MNIST 的 0–9），网络预测噪声：

$$L(\theta) = \mathbb{E}_{t,\mathbf{x}_0,c,\boldsymbol{\epsilon}}\Big[\big\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c)\big\|^2\Big]. \tag{2}$$

采样时喂入想要的 c，反向均值公式与无条件相同，只是 $\boldsymbol{\epsilon}_\theta$ 多了参数 c：

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t, c) = \frac{1}{\sqrt{\alpha_t}}\Big(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c)\Big). \tag{3}$$

为什么这样做？ 为什么前向不加条件、反向才加？ 加噪是「破坏信息」的固定物理过程，与「想生成哪个数字」无关；条件只告诉反向过程「往哪个数据流形去」。这对应贝叶斯视角：真正用到的是 $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ 的近似，而 $\mathbf{x}_0$ 的分布换成了条件分布 $q(\mathbf{x}_0 \mid c)$——训练时用带标签的样本估计它。

3.2 条件如何进入网络

本实验用**类别嵌入**：`nn.Embedding(11, emb_dim)`（下标 0–9 为数字，下标 10 为「无条件」空标签）。

$$\mathbf{e}(t, c) = \mathrm{MLP}(\mathrm{SinTime}(t)) + \mathrm{Embed}(c) \in \mathbb{R}^d, \tag{4}$$

再把 $\mathbf{e}$ 加到每个 ResBlock 的特征图上（广播到空间维）。这样「现在是第几步」与「要生成哪个数字」共用同一条条件通道。

3.3 引导（Guidance）：从 CG 到 CFG

3.3.1 为什么还要「引导」？——纯条件模型的不足

有了 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c)$，直接用它做祖先采样（$w = 1$）在理论上就是从 $p_\theta(\mathbf{x}_0 \mid c)$ 采样。但实践中常见两类现象：

1. **条件偏弱：** 网络容量与训练步数有限时，$p_\theta(\mathbf{x} \mid c)$ 往往比真实的 $p(\mathbf{x} \mid c)$ 更「宽」，会混入其他类的特征——本实验 $w = 1$ 时第 5 行偶尔画成 6 状、第 2 行偶尔出现 1/8 状（图 10）；
2. **多样但类别身份模糊：** 样本分布覆盖面广（多样性高），但其中一部分「既像 3 又像 8」，按类别看不够典型。

我们想要一个旋钮：牺牲一点多样性，换取更高的**类别保真度**（fidelity）——即让样本 $\mathbf{x}$ 的后验 $p(c \mid \mathbf{x})$ 更尖锐、更确信是 c。这就是 guidance 的目标：

$$\tilde{p}_w(\mathbf{x} \mid c) \;\propto\; p(\mathbf{x} \mid c)\, p(c \mid \mathbf{x})^{\,w-1} \;\propto\; p(\mathbf{x})\, p(c \mid \mathbf{x})^{\,w}. \tag{5}$$

$w = 1$ 时就是原条件分布；$w > 1$ 时，「分类器越确信是 c 的区域」被放大。

为什么这样做？ 温度类比。 语言模型里把 softmax 温度调低，会让高概率词更突出、输出更「确定」但更单调；式 (5) 对 $p(c \mid \mathbf{x})$ 做的是同一件事（p^w 相当于温度 $1/w$）。GAN 里的截断技巧（truncation trick）同样是「多样性换质量」。所以 guidance 不是让模型「更正确」，而是在保真度—多样性曲线上选一个点；FID 通常在中等 w 处最好，IS / 分类准确率随 w 单调升高。

3.3.2 Classifier Guidance（CG）：用外部分类器的梯度

Dhariwal & Nichol（2021，arXiv:2105.05233）的思路是对式 (5) 取对数再对 $\mathbf{x}_t$ 求梯度（score）。由贝叶斯公式 $p(\mathbf{x}_t \mid c) = p(c \mid \mathbf{x}_t)p(\mathbf{x}_t)/p(c)$，且 $p(c)$ 与 $\mathbf{x}_t$ 无关：

$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t \mid c) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(c \mid \mathbf{x}_t). \tag{6}$$

扩散模型的噪声预测与 score 的关系是 $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t) \approx -\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)/\sqrt{1-\bar{\alpha}_t}$（讲义 score 一节）。把分类器项乘上引导强度 s 后换回 $\boldsymbol{\epsilon}$ 记号：

$$\tilde{\boldsymbol{\epsilon}}(\mathbf{x}_t, t, c) = \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) - \sqrt{1-\bar{\alpha}_t}\; s\, \nabla_{\mathbf{x}_t} \log p_\phi(c \mid \mathbf{x}_t). \tag{7}$$

它对应的目标分布是 $p(\mathbf{x}_t)\, p_\phi(c \mid \mathbf{x}_t)^s$，即式 (5) 中 $w = s$。若底座本身已是条件模型 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c)$，则对应 $p(\mathbf{x}_t \mid c)\, p_\phi(c \mid \mathbf{x}_t)^s$，相当于 $w = 1 + s$。

CG 的代价：

- **要额外训练一个「噪声感知」分类器** $p_\phi(c \mid \mathbf{x}_t, t)$：普通 MNIST 分类器只见过干净图，在 t 很大的 $\mathbf{x}_t$ 上输出没有意义，必须在加噪数据上（输入含 t）重新训练；
- **每步要对分类器反向传播**求输入梯度，采样成本与显存都增加；
- **梯度可能「对抗」：**分类器的输入梯度容易指向人眼不可见的微小扰动（类似对抗样本），能提高分类器分数却不一定改善图像；
- 引导质量受限于分类器本身，换一种条件（如文本）就要再训一个对应的「分类器」。

3.3.3 从 CG 推出 CFG：把分类器「藏」进生成模型

Ho & Salimans（2022，arXiv:2207.12598）注意到，式 (6) 可以反过来用：隐式分类器的梯度等于两个生成模型 score 之差，

$$\nabla_{\mathbf{x}_t} \log p(c \mid \mathbf{x}_t) = \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t \mid c) - \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t) \approx -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \Big(\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c) - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \emptyset) \Big). \tag{8}$$

代入式 (7)（底座取无条件 $\boldsymbol{\epsilon}_\emptyset$，强度记作 w），$\sqrt{1-\bar{\alpha}_t}$ 恰好约掉：

$$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \emptyset) + w\big(\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c) - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, \emptyset)\big). \tag{9}$$

w 与 s 的对应：以无条件模型为底座时 $w = s$；Ho & Salimans 原文写成 $(1+w')\boldsymbol{\epsilon}_c - w'\boldsymbol{\epsilon}_\emptyset$，即 $w = 1 + w'$。本教案统一用式 (9) 的 w：$w = 1$ 为纯条件。

一个网络同时学 $\boldsymbol{\epsilon}_c$ 与 $\boldsymbol{\epsilon}_\emptyset$：训练时以概率 $p_{\text{uncond}} = 0.1$ 把标签换成空标签 $\emptyset$（代码中下标 10，`nn.Embedding(11,128)` 的第 11 行）。于是同一组参数在 90% 的样本上学条件 score、10% 上学无条件 score，采样时只需换输入标签就能得到两者。

为什么这样做？ 为什么丢弃率取 0.1 左右？太小（如 0.01）则无条件分支训练样本太少，$\boldsymbol{\epsilon}_\emptyset$ 不准，差值 $\boldsymbol{\epsilon}_c - \boldsymbol{\epsilon}_\emptyset$ 噪声大；太大（如 0.5）则条件分支少了一半数据，$w = 1$ 的质量下降。原文在 0.1–0.2 之间效果相近，0.1 是常用折中。
为什么叫「无分类器」？引导方向 $\boldsymbol{\epsilon}_c - \boldsymbol{\epsilon}_\emptyset$ 由生成模型自己给出，不需要外部分类器、不需要对输入求梯度——只是多一次前向，且没有对抗梯度问题。

3.3.4 w 的作用与本实验的实测

- $w = 0$：$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\emptyset$，完全忽略 c，即无条件生成（随机数字）；
- $w = 1$：纯条件模型，$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_c$；
- $w > 1$：沿「条件减无条件」方向外推，对应式 (5) 的锐化。

本实验同一模型、同一随机种子下（`runs/final/samples_g1.png`、`samples_g1.5.png`、`samples_g3.png`，图 10），用小 CNN 分类器（测试集 98.83%）判卷、每类 30 张：$w = 1$ 总体 **95.0%**（最弱为数字 5：26/30），$w = 1.5$ 为 **100%**（300/300），见表 2。肉眼看 $w = 3$ 的网格笔画更粗、同一行更「整齐划一」，多样性明显下降。

为什么这样做？ **大 w 的副作用与 $\hat{\mathbf{x}}_0$ 裁剪。**外推使 $\hat{\boldsymbol{\epsilon}}$ 的幅度超出训练时见过的范围，由它算出的 $\hat{\mathbf{x}}_0 = (\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\hat{\boldsymbol{\epsilon}})/\sqrt{\bar{\alpha}_t}$ 容易跑出 $[-1, 1]$，表现为过饱和、笔画发胖。本实验采样时先把 $\hat{\mathbf{x}}_0$ 裁剪到 $[-1, 1]$ 再算后验均值，这能防止像素「爆掉」，但也会把过度外推「压平」——所以大 w 下看到的是粗而整齐的数字，而不是噪点。修复前的小模型在 $w = 3$ 且未裁剪时出现过整行全黑，就是这种外推失控的极端情况。

3.3.5 对照：没有类别条件时（$w = 0$ / null 标签）

若采样时始终喂空标签 $\emptyset$（代码中 $c = 10$），则 $\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\emptyset$，完全忽略想要的数字——这就是式 (9) 里 $w = 0$ 的情形。用同一份 EMA 权重（`runs/final/ckpt_ema.pt`）实测采样 100 张（10×10 网格，耗时 144.7 s）：

- 样本仍然是清晰的手写数字（模型学到了 $p(\mathbf{x})$）；
- 但各类别混在每一行——行标签只是排版，不是条件（图 1）；
- 对比同权重下 $w = 1$（纯条件，分类器 95.0%）与 $w = 1.5$（CFG，100%）：没有条件就无法「指定生成数字 3」。

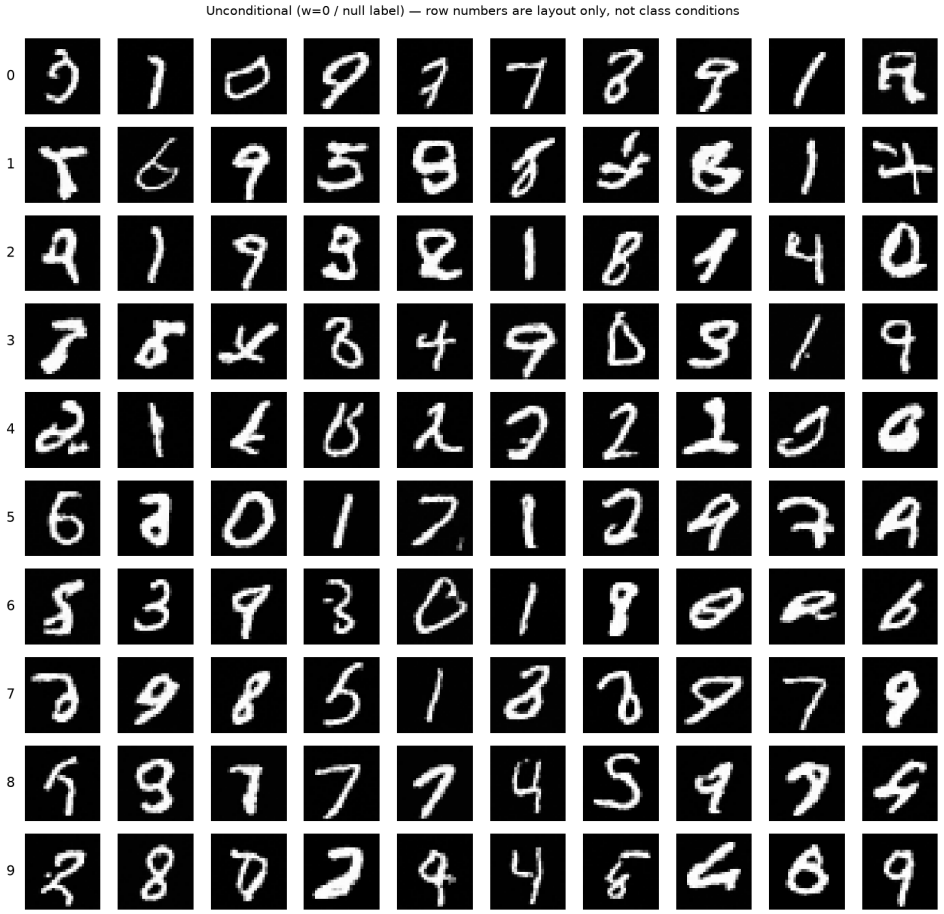


图 1　无条件采样（null 标签，$w = 0$ 等价）。标题已注明：左侧行号仅排版。肉眼可见各行混有 0–9。

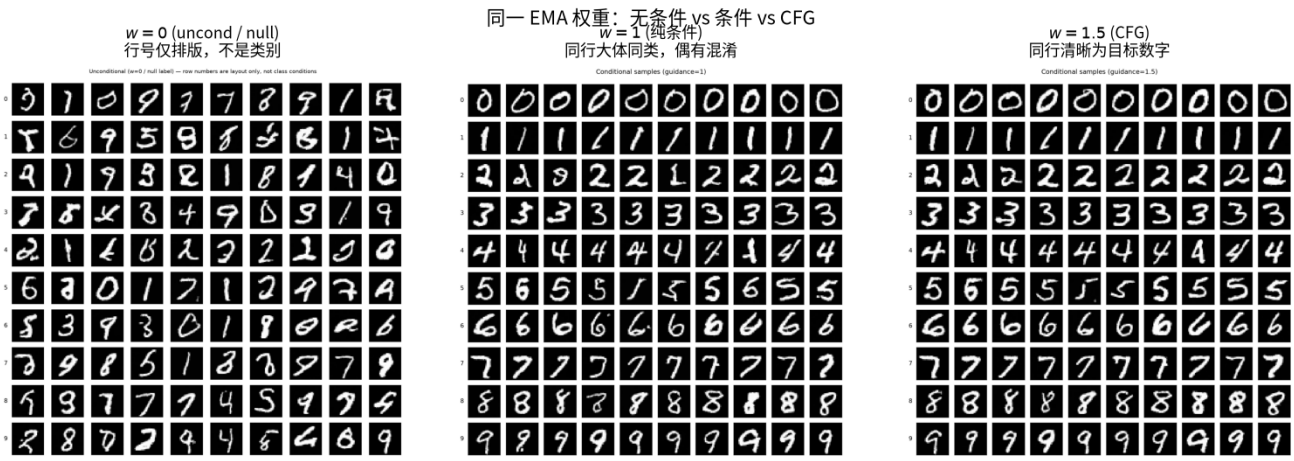


图 2　同一 EMA 权重下 $w = 0$ / 1 / 1.5 对照。无条件：数字像 MNIST 但类别随机；条件与 CFG：按行对齐目标数字。

3.3.6　CG 与 CFG 对比

	Classifier Guidance（CG）	Classifier-Free Guidance（CFG）
引导方向	$\nabla_{\mathbf{x}_t} \log p_\phi(c \mid \mathbf{x}_t)$，外部分类器的输入梯度	$-(\boldsymbol{\epsilon}_c - \boldsymbol{\epsilon}_\emptyset)/\sqrt{1-\bar{\alpha}_t}$，隐式分类器
额外模型	需另训噪声感知分类器 $p_\phi(c \mid \mathbf{x}_t, t)$	无；同一网络 + null 标签
训练改动	扩散模型不变	训练时以 $p \approx 0.1$ 丢标签
每步采样代价	1 次扩散前向 + 分类器前向与反向	2 次扩散前向（可拼成一个 batch）
风险	对抗性梯度、依赖分类器质量	大 w 过饱和、多样性下降
可扩展性	每种条件要一个分类器	任何条件（类别 / 文本 / 图像）通用
代表工作	ADM（Dhariwal & Nichol 2021）	GLIDE、Imagen、Stable Diffusion 等

3.3.7　代码对照（`classroom_cond_mnist.py`）

```python
# L191-192 training: drop label -> null (10) with p = 0.1
drop = torch.rand(yb.size(0), device=dev) < 0.1
yb = torch.where(drop, torch.full_like(yb, 10), yb)
# L105-110 sampling: cond + uncond in ONE batched forward
if guidance == 1.0:
    e = net(x, t, labels)                                   # plain conditional
else:
    e2 = net(torch.cat([x, x]), torch.cat([t, t]), torch.cat([labels, null]))
    e_c, e_u = e2.float().chunk(2)
    e = e_u + guidance * (e_c - e_u)                        # eq. (CFG)
# L114 then clip x0_hat to [-1, 1] before the posterior mean
```

把条件与无条件输入沿 batch 维拼接，一次前向就得到 $\boldsymbol{\epsilon}_c$ 与 $\boldsymbol{\epsilon}_\emptyset$，在 GPU 上比两次调用更省时；$w = 1$ 时直接跳过无条件分支。

四、模型架构详解

本节全部由 `train_cond_mnist.py` 中的 `CondUNet`（`base=48, emb_dim=128`）实例化后逐模块统计得到：输入 $(B, 1, 28, 28)$，参数合计 **2,139,713**（与训练日志一致）。采样时使用的是同结构的 EMA 副本（参数逐元素滑动平均，不增加层）。

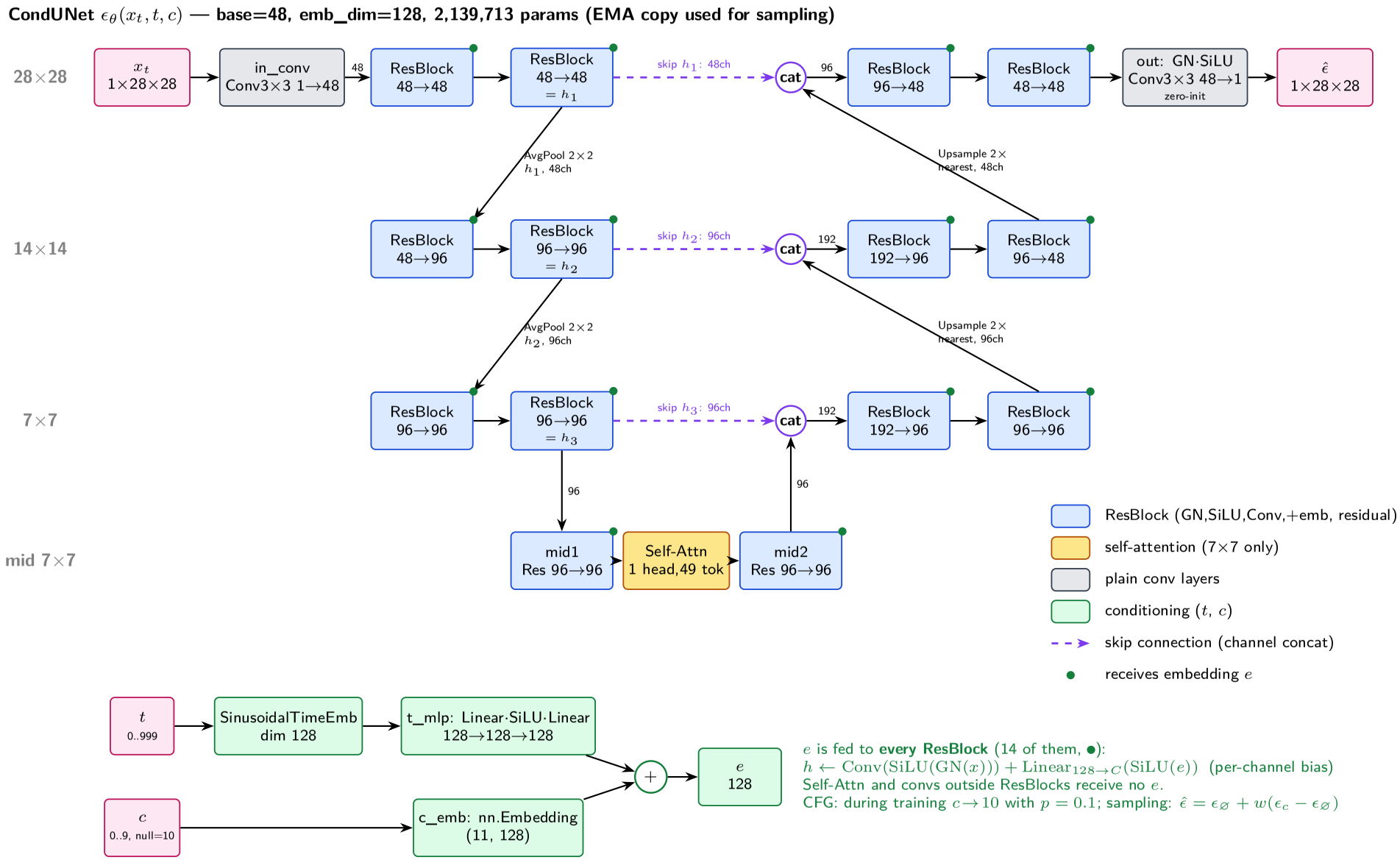


图 3　CondUNet 总体架构 $\epsilon_\theta(\mathbf{x}_t, t, c)$。蓝：ResBlock（右上角绿点表示注入了条件向量 $\mathbf{e}$）；黄：7×7 自注意力；紫色虚线：跳连（通道拼接）；绿：时间 / 类别条件通路。下采样为 AvgPool 2×2（作用在每级第二个块的输出 h_ℓ 上），上采样为最近邻 2×。

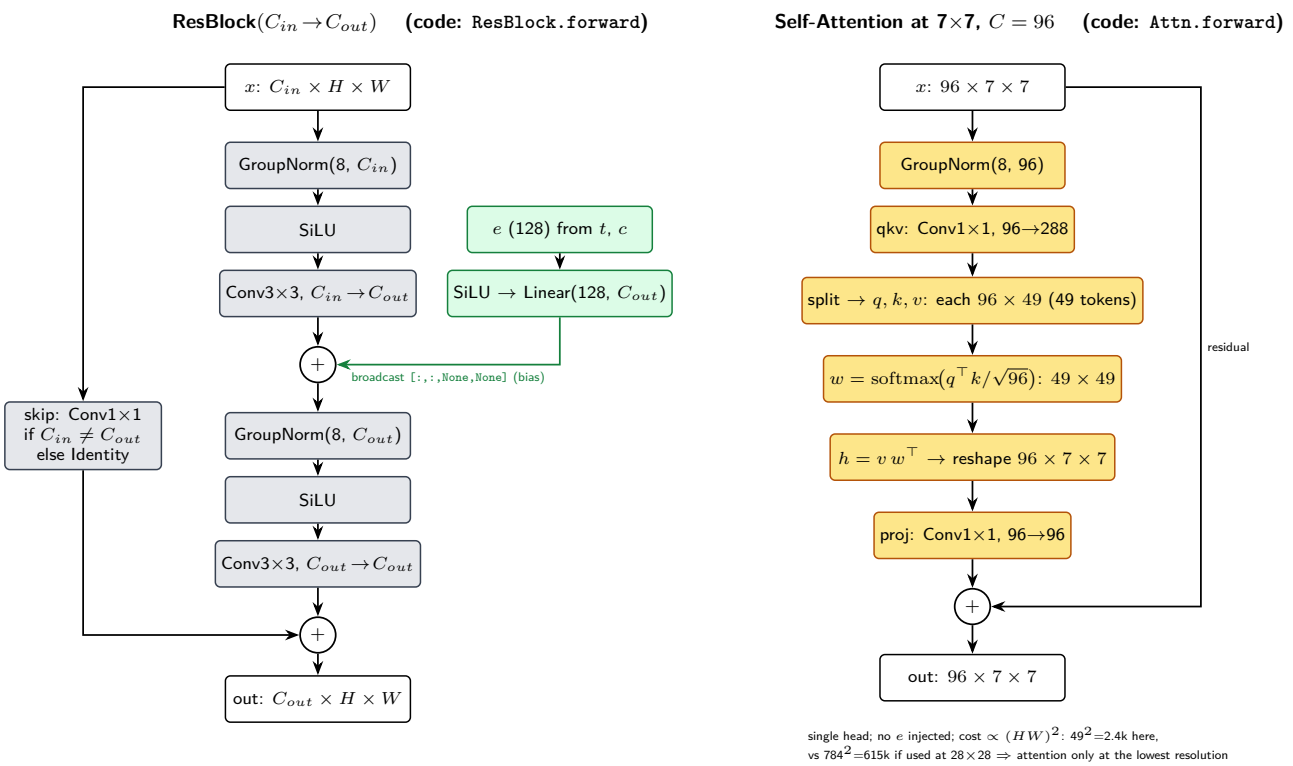


图 4　局部放大。左：ResBlock——条件向量经 SiLU 与 Linear(128 → C_{out}) 后作为逐通道偏置加在第一个卷积之后（不是 scale-shift）；残差支路在通道数变化时用 1×1 卷积。右：单头自注意力（只在 7×7 使用，不接收 e）。

4.1　逐层形状与参数（实测）

阶段	模块（代码名）	结构	输出形状	参数
条件	`t_emb`	正弦时间嵌入	128	0
	`t_mlp`	Linear·SiLU·Linear	128	33,024
	`c_emb`	Embedding(11,128)	128	1,408
Stem	`in_conv`	Conv3×3, 1→48	48 × 28 × 28	480
编码 28	`d1.0, d1.1`	ResBlock 48→48 ×2	48 × 28 × 28	95,904
编码 14	`d2.0, d2.1`	ResBlock 48→96, 96→96	96 × 14 × 14	320,832
编码 7	`d3.0, d3.1`	ResBlock 96→96 ×2	96 × 7 × 7	357,696
瓶颈	`mid1`	ResBlock 96→96	96 × 7 × 7	178,848
	`attn`	单头自注意力	96 × 7 × 7	37,440
	`mid2`	ResBlock 96→96	96 × 7 × 7	178,848
解码 7	`u3.0, u3.1`	cat(192)→96, 96→96	96 × 7 × 7	459,360
解码 14	`u2.0, u2.1`	cat(192)→96, 96→48	48 × 14 × 14	353,952
解码 28	`u1.0, u1.1`	cat(96)→48, 48→48	48 × 28 × 28	121,392
输出	`out`	GN·SiLU·Conv3×3 48→1	1 × 28 × 28	529
合计（`sum(p.numel())`）				**2,139,713**

单个 ResBlock 参数示例：96→96 为 178,848（两个 3×3 卷积各 83,040，嵌入投影 12,384，两个 GroupNorm 各 192）；解码首块 192→96 为 280,512（含 1×1 跳连卷积）。参数最多的是 14/7 分辨率的解码端，因为拼接使输入通道翻倍。

4.2 设计选择：为什么这样搭

为什么这样做?

- **多尺度（28→14→7）：**去噪既要全局「这是个几」，又要局部笔画边缘；下采样让感受野迅速覆盖整个数字。
- **跳连拼接：**下采样会丢细节，解码端从编码端 h_1, h_2, h_3 直接拿回同分辨率特征，输出才能有锐利笔画；拼接（而非相加）让后续卷积自己决定如何融合。
- **每个 ResBlock 都注入 e：**同一个网络要处理从几乎干净（$t \approx 0$）到纯噪声（$t \approx T$）的输入，每一层都需要知道「现在噪声多大、要画哪个数字」；只在输入处注入会被深层逐渐稀释。
- **时间 + 类别相加：**两者都映射到 128 维后相加，共用一条条件通道，实现最简单；类别表多一行 null（下标 10），训练时 10% 概率替换，使同一网络同时学会 $\boldsymbol{\epsilon}_c$ 与 $\boldsymbol{\epsilon}_\emptyset$，从而支持 CFG。
- **注意力只放在 7×7：**代价 $\propto (HW)^2$，7×7 只有 49 个 token（$49^2 \approx 2.4$k），28×28 则为 $784^2 \approx 61$ 万；在最低分辨率加一层就补上了全局感受野——这正是旧版小网络缺的。
- **GroupNorm：**不依赖 batch 统计，批量小或采样时（`eval`）行为与训练一致；8 组适配 48/96 通道。
- **输出卷积零初始化：**训练开始时 $\hat{\boldsymbol{\epsilon}} \equiv 0$，loss 从 ≈ 1 平稳下降，不会被随机输出干扰。
- **EMA：**采样用参数的指数滑动平均（衰减 0.999），相当于对最近约 1000 步的权重做平均，样本更平滑稳定；它不改变结构，只多存一份权重。

4.3 公式与模块对照

公式中的量	网络中的位置
$\mathbf{x}_t \in \mathbb{R}^{1\times28\times28}$	`forward` 的 `x`，进入 `in_conv`
$t \in \{0, \dots, 999\}$	`t_emb`（正弦）→ `t_mlp`
$c \in \{0..9\}$，$\emptyset = 10$	`c_emb = nn.Embedding(11,128)`
$\mathbf{e}(t, c)$	`emb = t_mlp(t_emb(t)) + c_emb(c)`
条件如何作用	每个 ResBlock：`h = conv1(...) + emb(silu(e))[:,:,None,None]`
$\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c)$	`out` 的输出，形状同 $\mathbf{x}_t$；训练目标式 (2)
$\hat{\boldsymbol{\epsilon}}$（CFG）	采样时调用两次网络（c 与 $\emptyset$），按式 (9) 组合

与双月实验对比：双月实验的 $\boldsymbol{\epsilon}_\theta$ 是输入 (x, y, t) 的小 MLP，没有空间结构、没有类别输入；这里的 U-Net 把同一个 $\boldsymbol{\epsilon}_\theta$ 换成了适合图像的卷积多尺度结构，数学目标完全不变。

五、教学准备

5.1 文件清单（`lab_mnist_cond.zip`）

文件	用途
`requirements.txt`	`torch, torchvision, numpy, matplotlib`
`train_cond_mnist.py`	完整训练 + 多 guidance 采样 + 保存 ckpt
`classroom_cond_mnist.py`	课堂单文件版：同一个 2.14M 参数 CondUNet，GPU 加速设置，可投影
`runs/final/ckpt_ema.pt`	已训练好的 EMA 权重（8.6 MB），免训练直接采样
`sample_by_digit.py`	指定单个数字采样，或重出 0–9 网格
`eval_classifier.py`	小 CNN 分类器给生成样本「判卷」（逐类准确率）
`runs/final/`	修复后推荐程（8 epochs）实测产出：网格、$\hat{\mathbf{x}}_0$ 检查、loss、分类评测
`runs/longer/`	修复前旧版（$T = 100$，15 epochs）产出，仅作反面对照
`figs/`	CFG 对比拼图等

5.2 环境与运行

沿用双月实验的 venv 即可，另装 torchvision：

```
# 若尚未安装（需与 torch 同为 CPU 轮）：
pip install torchvision --index-url https://download.pytorch.org/whl/cpu
cd lab_mnist_cond
# 推荐程（默认参数即此设定；本机实测训练 27.5 分钟 + 采样约 7 分钟）
python train_cond_mnist.py --epochs 8 --T 1000 --base 48 --device cpu --out runs/final
# 重出 10x10 网格 / 指定生成数字 3（自动使用 EMA 权重）
python sample_by_digit.py --ckpt runs/final/ckpt.pt --guidance 1.5
python sample_by_digit.py --ckpt runs/final/ckpt.pt --digit 3 --guidance 1.5 --n 16
# 用分类器给生成样本打分（首次会先训一个小 CNN，约 2 分钟）
python eval_classifier.py --ckpt runs/final/ckpt.pt --guidance 1.5 --n 30
```

MNIST 首次运行会自动下载到 `./data/`（torchvision 开源数据）：https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html。

实测记录（本机 8 核 CPU，PyTorch 2.14.1+cpu，未经挑选）。

- 模型：`CondUNet`（每级 2 个 ResBlock，28→14→7，7×7 处单头自注意力），**2,139,713** 参数；EMA 衰减 0.999；AdamW，lr 8×10^{-4} + 余弦退火；batch 128。
- 日程：线性 β：$10^{-4}\to0.02$，$T=1000$，$\bar\alpha_T\approx4.0\times10^{-5}$。
- 训练 8 epochs：**1649.2 s（27.5 分钟）**，约 205 s/epoch；末 50 步 loss = **0.0220**（逐轮：0.0293, 0.0259, 0.0243, 0.0230, 0.0226, 0.0221, 0.0215）。
- 采样：$\hat{\mathbf{x}}_0$ 检查 + 3 张 10×10 网格（$w=1, 1.5, 3$）共 **424.9 s**。
- 结果：100% 的行都是清晰可读的目标数字（图 5）；分类器判卷见表 2。

本实验第一版的 bug（已修复，留作反面教材）。第一版用 $T=100$、$\beta:10^{-4}\to0.02$，loss 停在 ≈0.054，但生成的「数字」完全认不出（图 6 左）。逐项排查后：

1. **主因：$\bar\alpha_T\approx0.36$，采样起点错位。**此时真实 $\mathbf{x}_T=0.6\,\mathbf{x}_0+0.8\,\boldsymbol{\epsilon}$；MNIST 背景像素 $=-1$，所以训练见过的 $\mathbf{x}_T$ 背景均值约 -0.6。采样却从 $\mathcal{N}(\mathbf{0},\mathbf{I})$ 出发——网络在 $t=T$ 附近从未见过这种输入，整条反向链从一开始就偏离，最后得到糊团、甚至整行全黑。双月实验数据已标准化（均值约 0），所以同样的日程「凑合能用」，换到图像就露馅。修复：$T=1000$，$\bar\alpha_T\approx4\times10^{-5}$。
2. **次因：网络太小（0.26M、每级 1 个块、无全局感受野）。**只修日程、3 epochs 时样本已成笔画但仍有空白行。升级为 2.1M 参数 U-Net + 7×7 注意力 + EMA 后 8 epochs 即清晰。
3. **采样细节：**先算 $\hat{\mathbf{x}}_0=(\mathbf{x}_t-\sqrt{1-\bar\alpha_t}\hat{\boldsymbol{\epsilon}})/\sqrt{\bar\alpha_t}$ 并裁剪到 $[-1,1]$，再用后验均值 $\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t,\hat{\mathbf{x}}_0)$ 与后验方差 $\tilde\beta_t$；不裁剪时它与 $\boldsymbol{\epsilon}$ 形式均值完全相等，裁剪则防止 CFG 外推把像素推出数据范围。

排查中确认无误的项：数据缩放 $[-1,1]$ 与保存时 $(x+1)/2$ 一致、t 下标训练/采样都是 0-based、最后一步不加噪、CFG 公式与 null 标签 $=10$、U-Net 跳连形状、保存图像未二次归一化。**教训：loss 下降 $\neq$ 采样正确**——loss 只检查「单步去噪」，不检查「采样起点是否与训练分布一致」。必做的健全性检查：$\hat{\mathbf{x}}_0$ 重建图（图 7）+ 直接看网格。

六、课堂流程

环节 A ｜从无条件到有条件（20 分钟）

教师：回顾双月实验「只能随机生成」。提问：若想「只要数字 7」该怎么办？引出 $p(\mathbf{x}\mid c)$ 与 $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t,t,c)$。板书式 (2) 与嵌入公式。
学生：对照双月代码，标出「哪里多了 c」。
检查点：能说出前向不加 c、反向 / 网络要加 c。

环节 B ｜代码对照（25 分钟）

教师：投影 `classroom_cond_mnist.py`，按表 1 走读：Embedding、CFG drop、闭式加噪、CFG 采样公式。

学生：在第「$e = te(t) + ce(c)$」行旁注释「条件入口」；在 CFG 公式旁写出式 (9)。

环节 C ｜ GPU 上现场训练 + 穿插讲解（30 分钟）

教师：在 A100 / H200 上运行 `python classroom_cond_mnist.py --epochs 10`（推荐 8–20 epochs）。脚本先打印 `params=2,139,713`，每个 epoch 打印 loss、本轮耗时与累计耗时，最后打印 `TRAIN DONE` 与采样耗时——**以屏幕上的真实数字为准**；本教案给出的 GPU 时间（10 epochs 约 2–5 分钟、100 张 × 1000 步采样约 1 分钟内）只是估计。等待期间讲：现场打印 $\bar{\alpha}_T$ 并对比 $T = 100$ 时的 0.36；U-Net 为何适合图像、GroupNorm、为何归一化到 $[-1, 1]$。

兜底：时间不够或 GPU 不可用时，运行 `python classroom_cond_mnist.py --load runs/final/ckpt_ema.pt`，直接用已训练好的 EMA 权重采样（8 核 CPU 实测：100 张 × 1000 步、$w = 1.5$ 共 256.5 s，网格清晰可读）。

学生：观察 loss 是否下降；打开 `classroom_out/samples_g1.5.png` 与 `loss.png`。

GPU 使用提示。

- 设备自动选择（`--device auto`：有 cuda 用 cuda）；多卡机器用 `CUDA_VISIBLE_DEVICES=0 python classroom_cond_mnist.py ...` 给每位学生 / 每组固定一张卡。
- 默认 bf16 autocast（A100/H200 原生支持），`--no-amp` 可关闭对照；同时开启 TF32 与 `cudnn.benchmark`。
- 整个 MNIST（$60000 \times 28 \times 28$，约 188 MB float32）一次性放进显存，每步用随机下标切片，不经过 DataLoader，GPU 不会等数据。
- 默认 `--batch 256`，学习率按 $8 \times 10^{-4}\sqrt{\text{batch}/128}$ 缩放；可试 512。EMA 衰减 0.999 按步计，batch 越大每 epoch 步数越少，总 epochs 宜相应增加（如 batch 512 用 16–20 epochs）。
- `--compile` 可选开启 `torch.compile`（首次编译需额外时间，默认关闭以求稳妥）。

环节 D ｜条件采样与 CFG（20 分钟）

教师：展示图 5、6、10；用 `sample_by_digit.py --digit 3` 现场要一个「3」。讨论 $w = 1, 1.5, 3$ 的差别。

学生：验收清单打勾；讨论第八节问题。

七、板书公式

$$\text{(1) 条件训练：}\quad L = \mathbb{E}\left\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t, c)\right\|^2, \tag{10}$$

$$\text{(2) 条件入口：}\quad \mathbf{e} = \mathrm{MLP}(\mathrm{SinTime}(t)) + \mathrm{Embed}(c), \tag{11}$$

$$\text{(3) CFG 采样：}\quad \hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\emptyset + w(\boldsymbol{\epsilon}_c - \boldsymbol{\epsilon}_\emptyset),\quad \mathbf{x}_{t-1} = \tfrac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \tfrac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\hat{\boldsymbol{\epsilon}}\right) + \sigma_t \mathbf{z}, \tag{12}$$

$$\text{(4) 检查：}\quad \bar{\alpha}_T \approx 0\ (T = 1000)\ \text{才能从}\ \mathcal{N}(\mathbf{0}, \mathbf{I})\ \text{出发；}\quad \hat{\mathbf{x}}_0 = \tfrac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}}. \tag{13}$$

八、核心代码要点与对照表

课堂单文件版 `classroom_cond_mnist.py` 与 `train_cond_mnist.py` 使用逐行相同的 `CondUNet`（可互相加载权重）。下面只列出「条件相关」关键片段（行号对应 `classroom_cond_mnist.py`）：

```
self.c_emb = nn.Embedding(num_classes + 1, emb_dim)   # L65: 0..9 + 10=null (CFG)
emb = self.t_mlp(self.t_emb(t)) + self.c_emb(c)       # L84: time + class
h = h + self.emb(F.silu(emb))[:, :, None, None]       # L40: bias in every ResBlock
```

```
4
5  drop = torch.rand(yb.size(0), device=dev) < 0.1       # L191-192: drop labels
6  yb = torch.where(drop, torch.full_like(yb, 10), yb)
7
8  e2 = net(torch.cat([x, x]), torch.cat([t, t]), torch.cat([labels, null]))
9  e_c, e_u = e2.float().chunk(2)                        # L108-110: CFG
10 e = e_u + guidance * (e_c - e_u)
```

表 1　条件相关公式与代码对照（classroom_cond_mnist.py / train_cond_mnist.py）

公式 / 概念	代码位置
$\mathrm{Embed}(c)$，$c \in \{0..9, \emptyset\}$	L65 `nn.Embedding(11, 128)`；null 下标 $= 10$
$\mathbf{e}(t, c)$ 注入网络	L84 `emb = t_mlp(t_emb(t)) + c_emb(c)`；L40 ResBlock 内逐通道偏置
日程 $\bar{\alpha}_t$，$T = 1000$	L12–15
闭式 $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$	L195，四维广播 `[:,None,None,None]`
式 (2)	L197 `mse_loss(fwd(xt,t,yb), eps)`（bf16 autocast）
训练时 $c \leftarrow \emptyset$	L191–192 `torch.where(drop, 10, yb)`，$p = 0.1$
EMA 0.999	L201 `pe.lerp_(pm, 1-0.999)`
式 (9)	L108–110，条件/无条件拼成一个 batch 前向
反向一步	L114 估计并裁剪 $\hat{\mathbf{x}}_0$；L115–117 后验均值、方差 $\tilde{\beta}_t$；最后一步不加噪

九、预期结果（实测图）

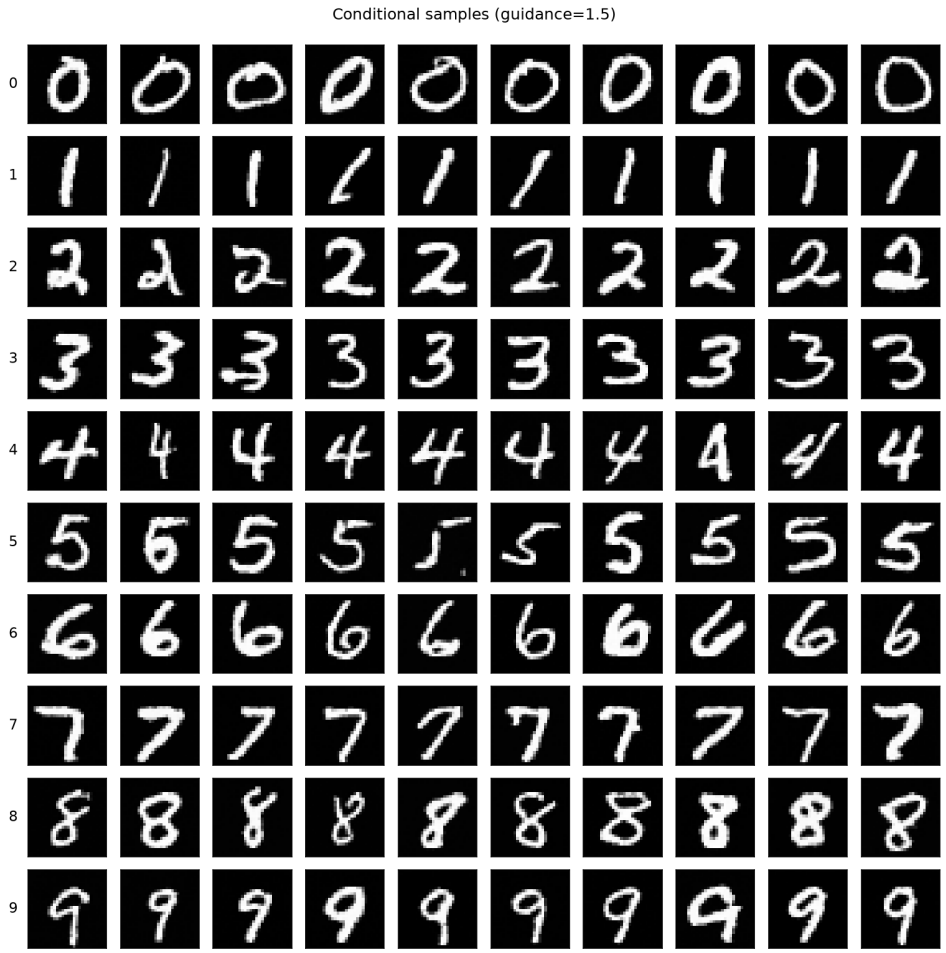


图 5　修复后推荐程（8 epochs，27.5 分钟，EMA 权重，$w = 1.5$）10×10 条件网格，各行为条件数字 0–9。未挑选。

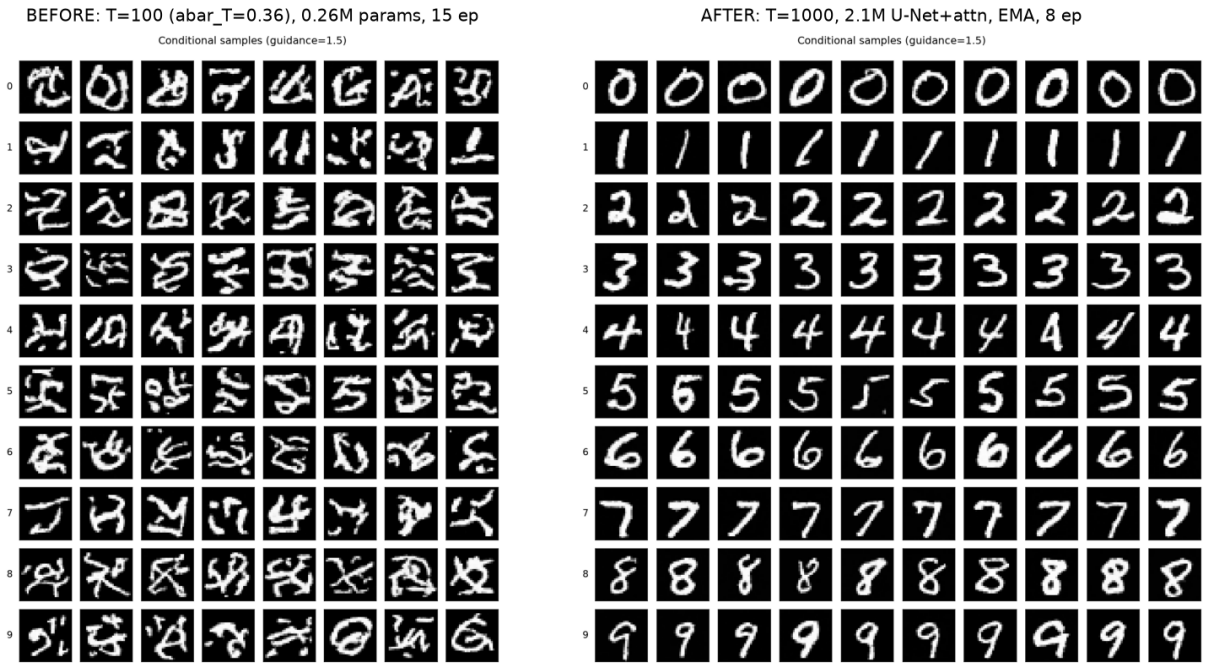


图 6　修复前（左：$T = 100$、0.26M 参数、15 epochs）与修复后（右）的同一可视化。左侧 loss 已平台化却认不出数字。

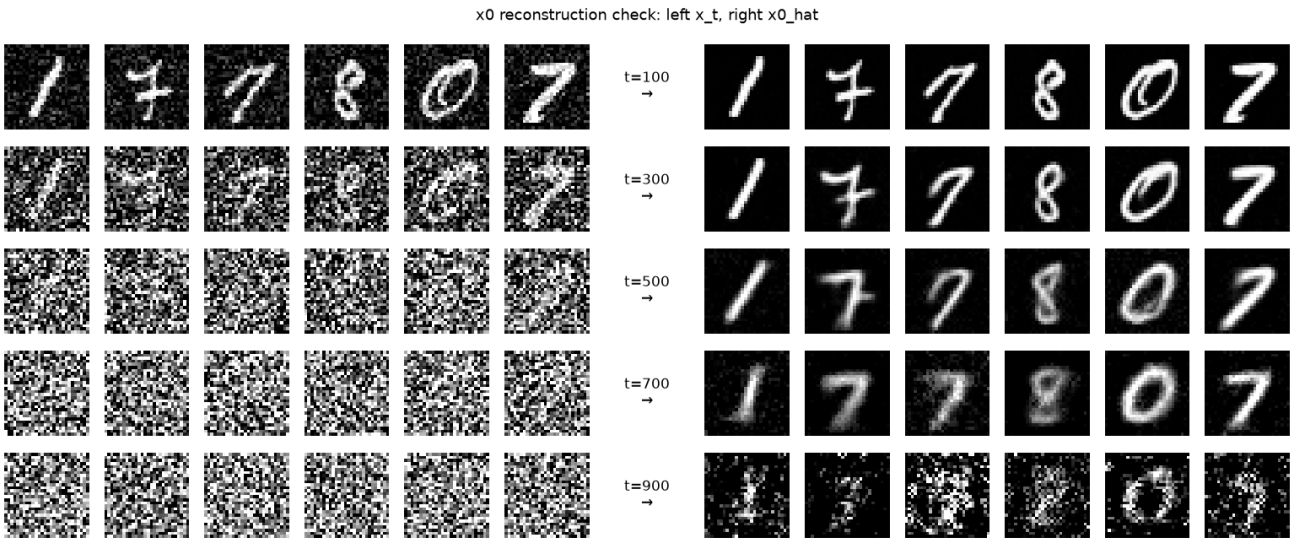


图 7　$\hat{\mathbf{x}}_0$ 重建检查：真实图加噪到 t（左），用预测噪声一步闭式估计 $\hat{\mathbf{x}}_0$（右）。$t \le 700$ 仍能认出原数字，说明网络与日程正确；$t = 900$ 时信息几乎被噪声抹掉，需多步采样。

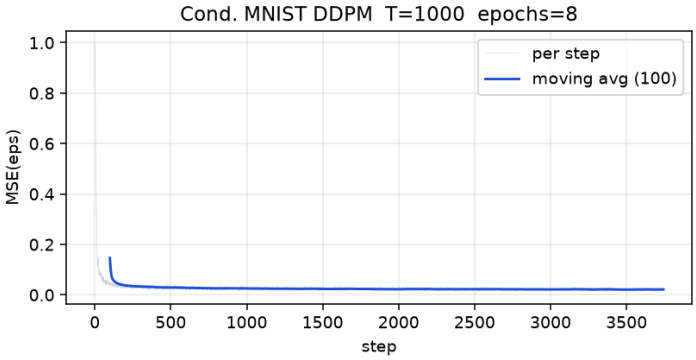


图 8　训练 loss（灰：逐步；蓝：100 步滑动平均），8 epochs 共 3744 步，末段约 0.022。注意 T 不同的 loss 不可直接比较。

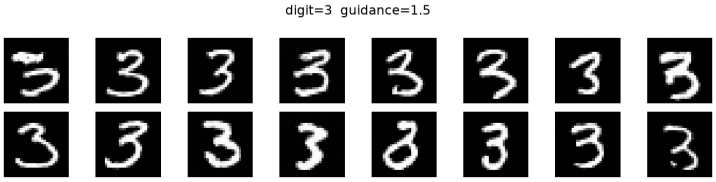


图 9　`sample_by_digit.py --digit 3 --guidance 1.5 --n 16`（修复后模型）。

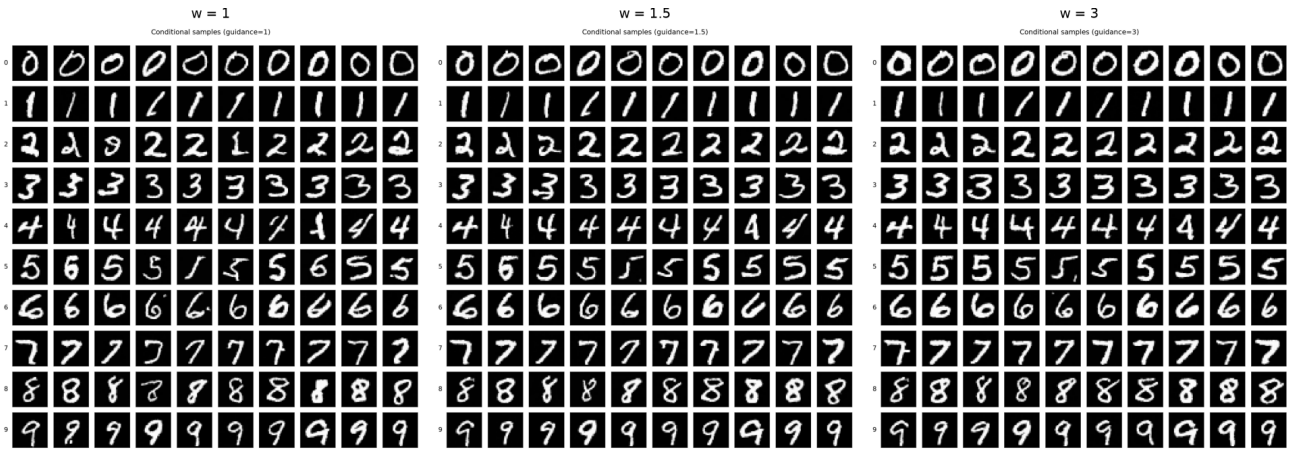


图 10　同一模型、同一随机种子下 $w = 1,\ 1.5,\ 3$。$w = 1$ 偶有类别混淆（如第 2 行出现 1/8 状、第 5 行出现 6 状），w 增大后更「典型」、笔画更粗、多样性下降。

表 2　分类器判卷：小 CNN（MNIST 测试集准确率 98.83%，训练 2 epochs / 45 s）对生成样本逐类判别（每类 30 张，EMA 权重，未挑选）。

w	0	1	2	3	4	5	6	7	8	9	总体
1	30/30	30/30	28/30	28/30	28/30	26/30	30/30	29/30	28/30	28/30	**95.0%**
1.5	30/30	30/30	30/30	30/30	30/30	30/30	30/30	30/30	30/30	30/30	**100%**

$w = 1$ 时最弱的是数字 5（26/30，常被画成 6 状），与图 10 的肉眼观察一致；$w = 1.5$ 的 300 张全部判对。采样耗时：$w = 1.5$ 300 张 1201 s，$w = 1$ 300 张 707 s（$w = 1$ 无需无条件前向）。

十、验收清单

- ☐ 打印的 $\bar{\alpha}_T$ 接近 0（本实验 4×10^{-5}）；loss 降到约 0.02 量级；
- ☐ $\hat{\mathbf{x}}_0$ 检查图中 $t \le 500$ 能认出原数字；
- ☐ 0–9 网格**每行至少 8/10 张肉眼可读且为目标数字**；
- ☐ 看过 $w = 0$ 无条件网格（图 1）：仍是数字，但类别混杂——理解「条件」的作用；
- ☐ `sample_by_digit.py --digit 3` 得到的图均为「3」（图 9）；
- ☐ 能指出条件入口那一行，并能解释 CFG 公式中 w 的作用；
- ☐ 能说明：本实验与双月实验共享哪些公式、多了什么。

十一、讨论与作业

1. 无条件模型能否「只要 7」？需要什么额外机制？
2. 若训练时从不丢掉标签（$p_{\text{uncond}} = 0$），CFG 还能用吗？为什么？
3. 把 guidance 设为 0，对应公式里什么？生成的还是指定数字吗？
4. **作业：**用 `--T 100` 重训（其余不变），复现图 6 左侧的失败，并打印 $\mathbf{x}_T$ 背景均值解释原因；或把条件从「类别」换成「奇偶」二分类，观察是否仍可控。

十二、常见问题

现象	原因与处理
所有行长得差不多	条件没喂进网络（忘了 `+ ce(c)`），或采样时 labels 全是同一值。
loss 平台化但数字认不出	先查 $\bar{\alpha}_T$：$T = 100$ 时 $\bar{\alpha}_T \approx 0.36$，采样起点与训练错位（本实验第一版的 bug）；再看 $\hat{\mathbf{x}}_0$ 检查图。
某几行整行全黑 / 空白	同上；或网络太小 + CFG 外推过度且未裁剪 $\hat{\mathbf{x}}_0$。
指定 3 却像噪声	训练太短、采样用了未训练的权重（忘加载 EMA / ckpt），或 guidance 为 0。
`Embedding` 报错 index 10	`Embedding` 大小须为 11（0..9+null）。
图像全白 / 全黑	忘了 Normalize 到 $[-1, 1]$，或保存前没做 `(x+1)/2`。
torchvision 与 torch 不匹配	用同一 CPU 轮安装：`pip install torchvision --index-url https://download.pytorch.org/whl/cpu`。
没时间训练	`classroom_cond_mnist.py --load runs/final/ckpt_ema.pt`，直接用已训练好的 EMA 权重采样。

十三、与双月教案、数学讲义的衔接

本实验	对应材料
前向闭式、ϵ-loss、祖先采样	双月教案第四–五节；讲义式 (2.4)、(4.9)、算法 1–2
重参数化构造 $\mathbf{x}_t$	讲义 §2.4；双月代码第 50 行同构
条件分布 $p(\mathbf{x} \mid c)$	讲义「反向匹配后验」在条件数据上的自然推广
CFG	Ho & Salimans 2022；与讲义 score 观点：引导 = 沿条件得分方向加大步长

建议顺序：双月无条件跑通 → 本条件 MNIST → 回到讲义第三章理解「反向一步为何像高斯」→ 课外 Diffusers 文生图教程。

十四、参考资料与链接

[1] J. Ho et al. Denoising Diffusion Probabilistic Models. https://arxiv.org/abs/2006.11239

[2] J. Ho, T. Salimans. Classifier-Free Diffusion Guidance. https://arxiv.org/abs/2207.12598

[3] P. Dhariwal, A. Nichol. Diffusion Models Beat GANs on Image Synthesis（Classifier Guidance）. https://arxiv.org/abs/2105.05233

[4] torchvision MNIST：https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html

[5] Hugging Face Diffusers —Conditional image generation：https://huggingface.co/docs/diffusers/using-diffusers/conditional_image_generation

[6] Diffusers —Classifier-Free Guidance 说明：https://huggingface.co/docs/diffusers/using-diffusers/classifier_free_guidance

[7] Diffusers 基础训练教程：https://huggingface.co/docs/diffusers/tutorials/basic_training

[8] PyTorch 安装：https://pytorch.org/get-started/locally/

[9] 前置教案：同目录上一份 `lab_2d_ddpm/diffusion_lab_lesson_plan.pdf`

[10] 配套数学讲义：`diffusion_math_teaching_notes_latex.pdf`

外链均于 2026-10-10 访问验证（HTTP 200）。